

SAGE

Contention-Aware Sensitivity Adjustment for Spin Lock Exiting

Sam Stowers, MS
Computer Science

The SAGE algorithm for optimizing CPU preemption in cloud systems through dynamic spinlock detection. Evaluation achieves up to 110% speedup on sensitive workloads, and a 13% average speedup in high contention.

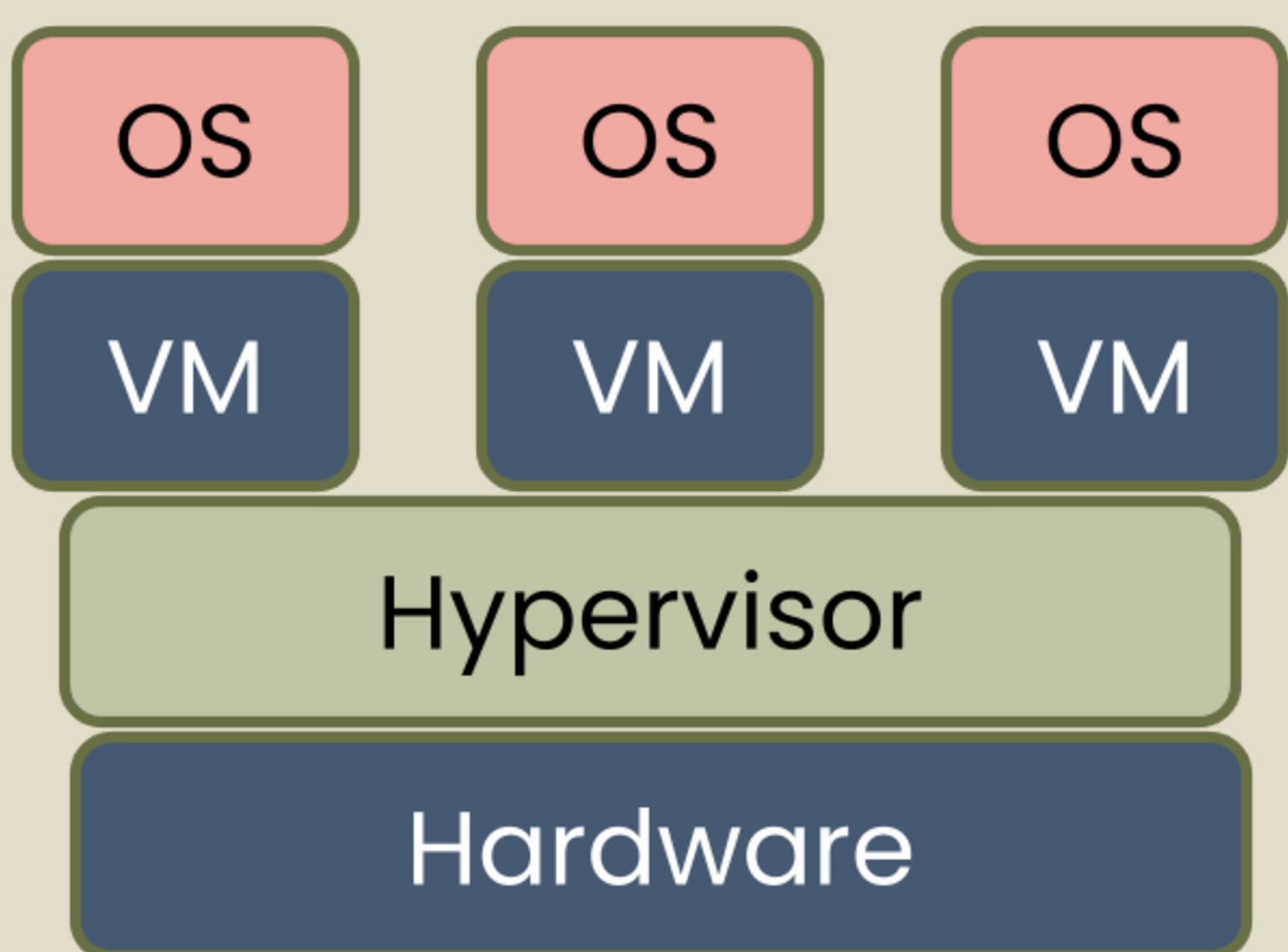


Figure 1: Virtualization Stack

A hypervisor manages a cloud server by deploying multiple VMs simultaneously onto the same hardware.

Excessive Spinning in Virtual Machines

Cloud systems deploy multiple guest operating systems (OS), each their own Virtual Machine (VM), onto cloud servers. Since each VM rarely utilizes their entire hardware allocation, cloud vendors subscribe more virtual hardware than physical hardware. This greatly boosts resource utilization at the cost of contention when the total guest resource usage exceeds what the server can actually provide.

vCPU Scheduling: When Virtual CPU (vCPU) demand exceeds the hardware capacity, each vCPU gets scheduled in a discrete time slice.

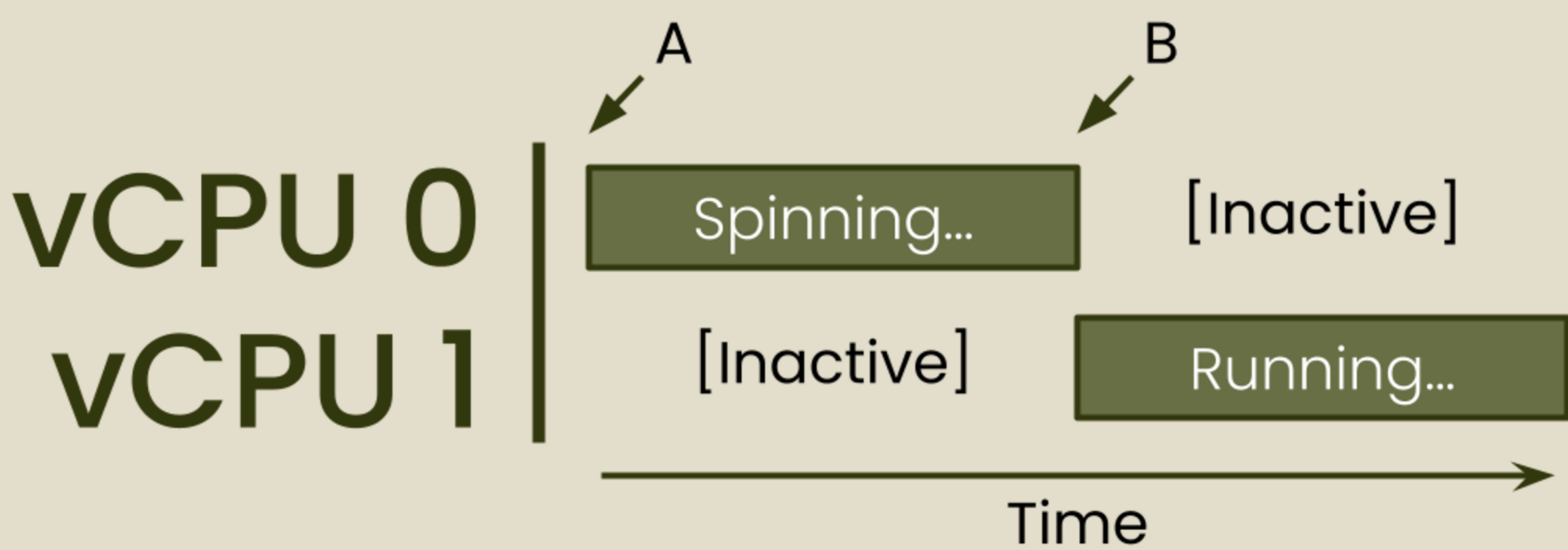


Figure 2: vCPU Discontinuity Problem

An example of how the hypervisor might schedule 2 virtual CPUs onto a single physical CPU

Consider Figure 2. In multi-threaded programs, a CPU often needs to wait for another CPU to finish work. If vCPU 0 requires vCPU 1 to finish before it can continue, the duration between point A and point B is **excessive spinning**. The earlier we can detect spinning, and the earlier we preempt vCPU 0 (i.e. at point B), the better the performance.

Using Hardware to Detect Spinlocks

Modern x86 hardware counts **pause** instructions built into each spin, and enough pauses incurs a **Pause Loop Exit (PLE)** Event. This lets the hypervisor intervene to deschedule the spinning vCPU. PLE acts as point B in Figure 2.

```
.spin:
    pause
    test dword [lockvar], 1
    jnz .spin
    jmp acquireLock
```

Figure 3: x86 spinlock instructions

PLE's effectiveness depends on correct spin detection, but current mechanisms with static sensitivity are inadequate. The ideal sensitivity can speed up every contended workload.

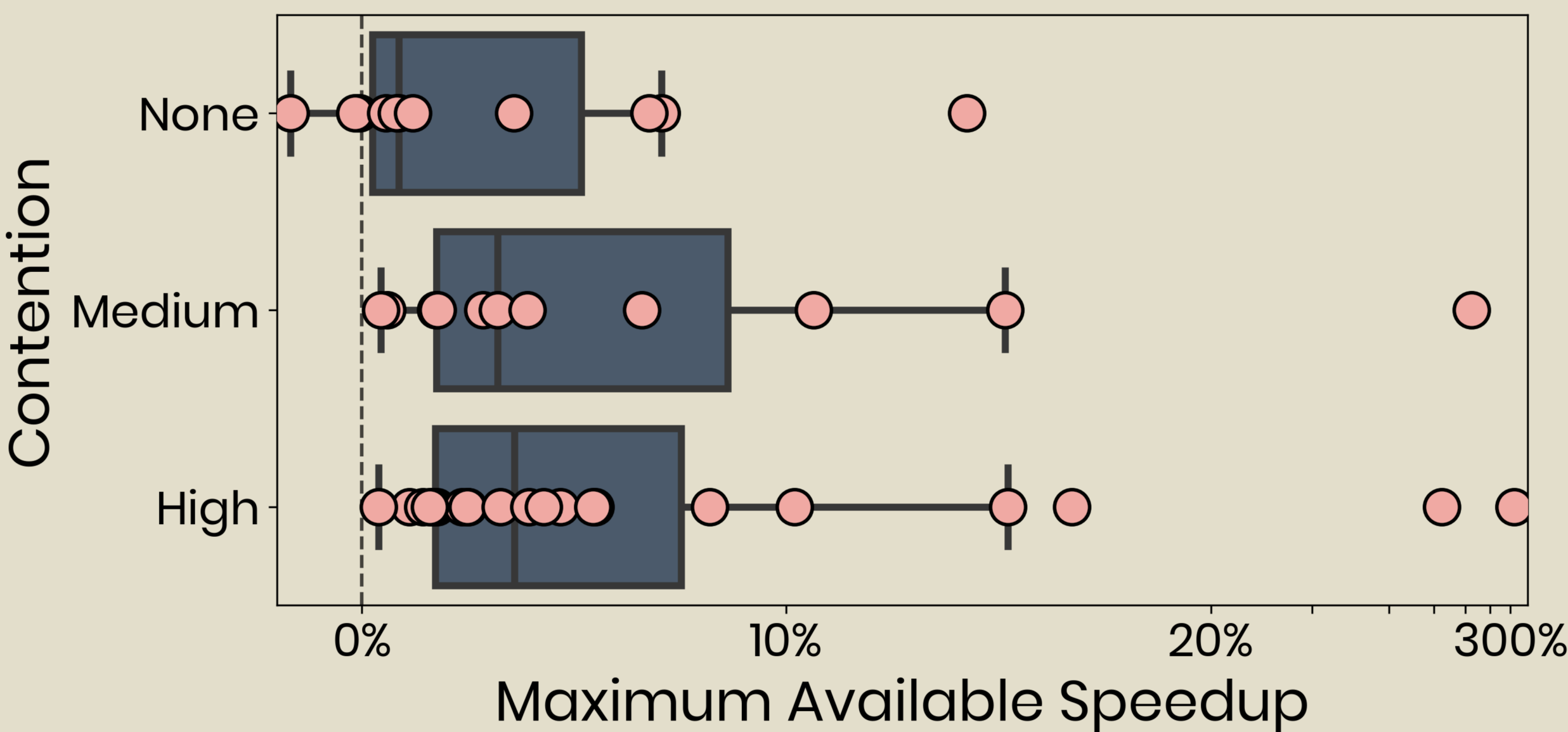


Figure 4: Theoretical Ideal Sensitivity Speedup vs Linux Default

40 different workloads all experience speedups with the ideal sensitivity value, however, that ideal value differs depending on the workload and contention level. A static parameter value will never achieve peak performance in all situations.

To address this I implemented

Sensitivity Adjusted Gap Exiting (SAGE)

The Design

Preemption is good. We want to encourage PLE as much as possible when contention is high.

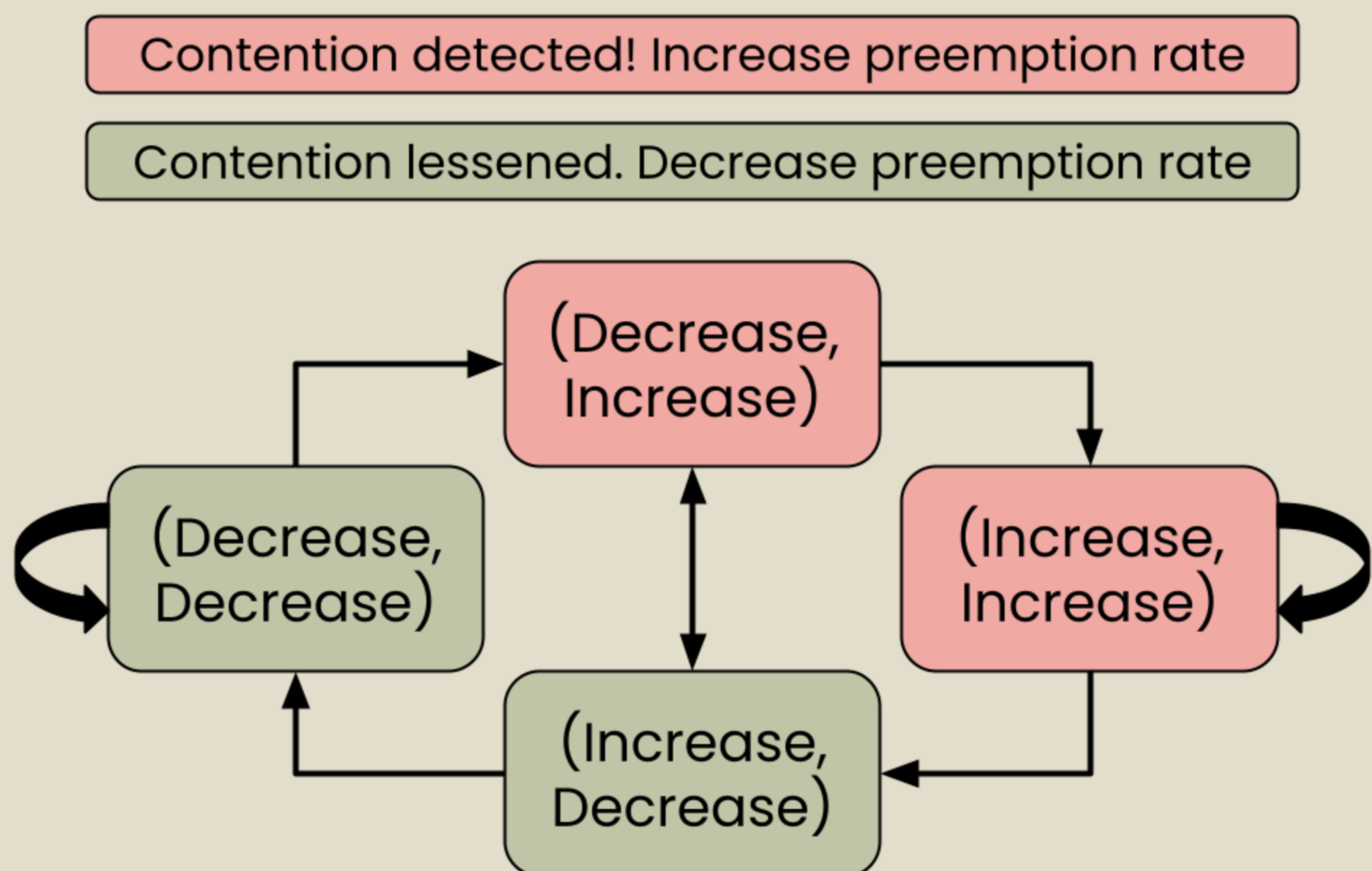


Figure 5: SAGE Contention State Machine

Sensors built into the vCPU descheduler and PLE handler track the rate of preemption through a 2-bit predictor. Sensitivity is adjusted according to the state change whenever a vCPU is rescheduled.

The Results

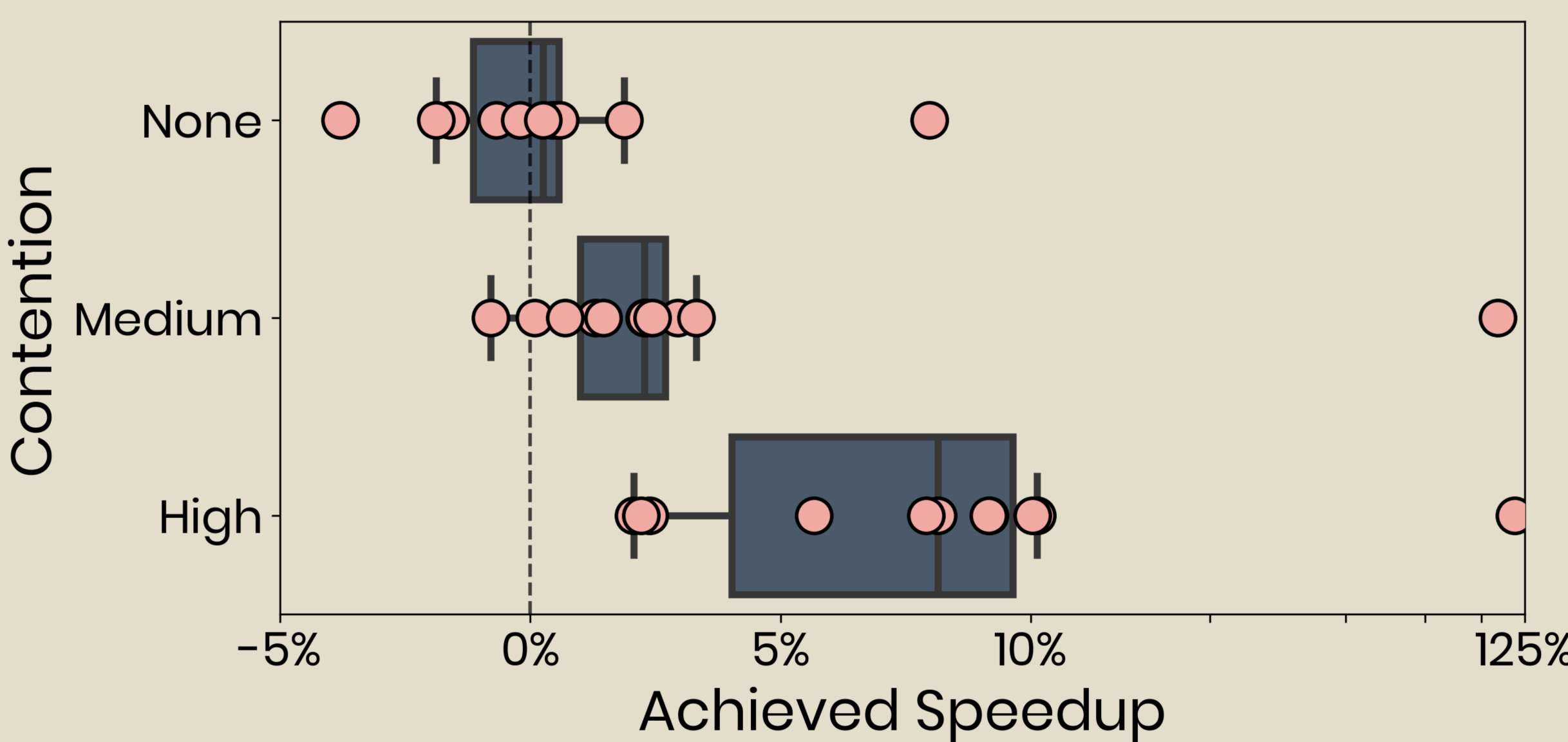


Figure 6: SAGE Performance vs Linux Default

SAGE achieves performance speedups across most medium contention workloads and all high contention workloads. Uncontested workloads exhibit minor overheads.

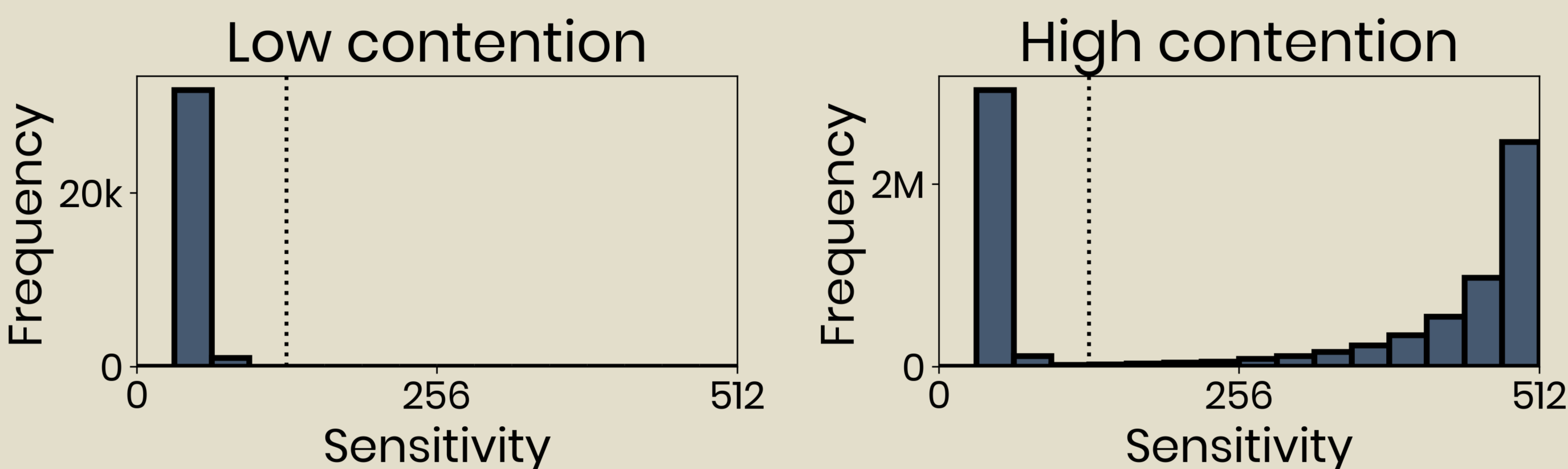


Figure 7: Sensitivity variability in the same benchmark (canneal) with different contention levels

Sensitivity adjusts accordingly during runtime. The same benchmark becomes more sensitive to preemption under high contention.

This work demonstrates current PLE mechanism insufficiency, the benefits of improved sensitivity adjustment, and an algorithm to achieve contention aware spinlock preemption. This lightweight modification can be deployed across cloud data centers to ease CPU bottlenecks in contended VMs.

Conclusion